The University of Azad Jammu and Kashmir, Muzaffarabad

Department of Software Engineering



Student Name	**Abdul Hadi Kiani**
Roll Number	**2025-SE-11**
Course Title	**Software Design and Architecture**
Course Code	**SEC-532**
Course Instructor	**Engr. Rubani Butt**
Program	**BSc Software Engineering**
Semester	**Fall 2026**
Session	**2025-2029**
Assignment No	**01**
Title	**Domain Modal**
Assigned Date	**October 22, 2026**
Due Date	**October 29, 2026**

Table of Contents

1. List of 10 Candidate Conceptual Classes 3
1. Student 3
2. Course 3
3. Professor 3
4. Department 3
5.Semester 3
6.Registration 3
7.Classroom 3
8. Schedule / TimeSlot 3
9. Transcript 3
10. Prerequisite 3
2. Attributes and Methods for Three Core Classes 3
A) Class: Student 3
B) Class: Course 3
C) Class: Registration 3
3. Main Abstraction Decisions 4
4. Invented Software Object 4

Individual Short Report: Conceptual Class Identification

1. List of 10 Candidate Conceptual Classes

1. Student
The primary actor who registers for courses.

2. Course
The central entity being offered and enrolled in.

3. Professor
Responsible for teaching a course section.

4. Department
Owns and manages a collection of courses.

5.Semester
Defines the time period in which registration occurs.

6.Registration
Represents the actual enrollment transaction/event.

7.Classroom
Physical location where the course is conducted.

8. Schedule / TimeSlot
Defines when a course class takes place.

9. Transcript
Maintains academic history and GPA of a student.

10. Prerequisite
A rule that controls eligibility for a course.

2. Attributes and Methods for Three Core Classes

A) Class: Student
Attributes: studentId, name, email, cgpa, enrolledCredits, department

Methods: registerForCourse(Course), dropCourse(Course), viewTranscript(), checkEligibility(Course)

B) Class: Course
Attributes: courseCode, title, creditHours, maxCapacity, currentEnrollment, description

Methods: addStudent(Student), removeStudent(Student), isFull(): boolean, getPrerequisites()

C) Class: Registration
Attributes: registrationId, registrationDate, status, grade

Methods: `confirmRegistration(), cancelRegistration(), calculateFee(), generateSlip()[Enrolled][Dropped][Waitlisted]

Abdul Hadi

3. Main Abstraction Decisions

I deliberately left out low-level implementation details to keep the model purely conceptual. For example, I did not include *User Login, Password, or Database Connection* because they are software implementation concepts, not real-world domain concepts. I also excluded *Fee Challan, University Bus, and Library* as they are related but not directly part of the core registration activity. I modeled **Schedule** as a separate class instead of just an attribute in Course, because a single course can have multiple time slots and rooms, which makes the system more flexible and realistic. The focus was kept on WHAT the system is about, not HOW it will be coded.

4. Invented Software Object

Invented Object: “RegistrationValidator”

This object does not exist in the real world but is extremely useful in software. Its purpose is to act as a controller/manager that performs all checks before a registration is confirmed. It will check for schedule conflicts, prerequisite completion, CGPA requirements, and maximum credit hour limits. This keeps the Student and Course classes clean and follows the Single Responsibility Principle.

5.Class Diagram

